



Jobly

Project Walkthrough & Technical Documentation

A complete architecture, workflow, and deployment reference for the Jobly workforce management platform — written so a new developer can understand the system and confidently build the next feature.

Generated July 21, 2026

Table of Contents

1. Executive Overview	4
2. System Architecture	4
3. Technology Stack	5
3.1 Frontend	5
3.2 Backend	7
3.3 Database & Managed Platform Services	7
3.4 Version control & CI	7
4. Public Marketing Site	8
Legacy jQuery integration	8
5. Portal Application	8
5.1 Page inventory	9
5.2 Role-permission matrix	11
5.3 Authentication flow	11
5.4 State management convention	12
5.5 UI library & the custom Select component	12
5.6 Onboarding wizard	12
5.7 Notification system	13
6. Backend Architecture	13
6.1 Request pipeline	13
6.2 Domain / route map	13
6.3 Auth & RBAC internals	15
6.4 Validation pattern	15
7. Database Schema	15
7.1 Tables defined in tracked migrations	15
7.2 Important gap — tables that exist live but aren't tracked	16

7.3 Key relationships	16
8. Core Business Workflows	16
8.1 Onboarding state machine	16
8.2 Timesheet state machine	17
8.3 Notification fan-out & dedup	17
9. Integrations	17
9.1 Email — Azure Communication Services	17
9.2 PDF / DOCX generation	18
9.3 Supabase Auth & Storage	18
10. Deployment & Infrastructure	18
10.1 Frontend — automatic	18
10.2 Backend — manual runbook	18
10.3 Environment configuration	19
10.4 Database migrations	20
11. How to Add a New Feature	20
Appendix A — Key File Reference Index	21
Appendix B — Glossary	22

1. Executive Overview

Jobly is a workforce management platform built for a staffing/consulting business: it combines a public marketing website with a full internal operations portal used to hire, onboard, staff, timesheet, invoice, and manage employees end-to-end. The codebase is a single monorepo containing both a public-facing site and a role-based internal application, plus a separate backend API and a managed Postgres database.

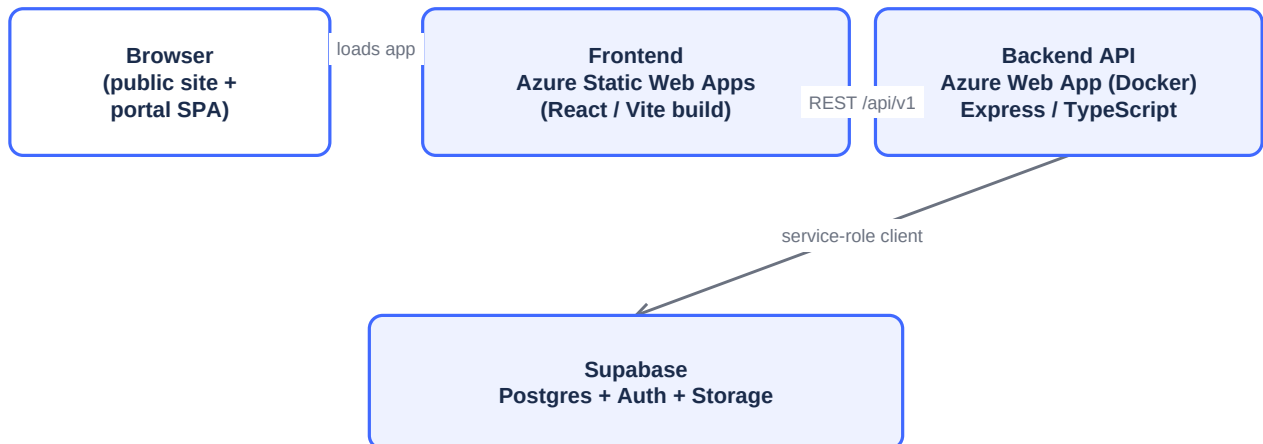
The system serves five distinct user roles, each with a different day-to-day job inside the portal:

Role	What they own in the system
Admin	Full access across every module — the operational super-user role.
HR	Owns hiring, onboarding review/approval, employee records, documents, performance reviews, benefits enrollment.
Operations	Owns staffing assignments, workforce scheduling, timesheet/attendance oversight.
Finance	Owns client invoicing, payments, budgets, expense and cash-flow reporting.
Employee	Self-service only — own profile, own timesheets, leave requests, expenses, documents, reviews.

This document is organized so a developer new to the project can start at the top (architecture), drill into the frontend and backend in detail, understand the database and core workflows, and finish with a concrete runbook for adding a new feature the same way the existing codebase does it.

2. System Architecture

Jobly is a classic three-tier web application, split into two independently deployed pieces plus a managed data platform:



Layer	Detail
Frontend hosting	Azure Static Web Apps — static bundle only, no server-side code. Auto-deployed on every push to main via GitHub Actions.

Layer	Detail
Backend hosting	Azure Web App for Containers — a Docker image running the Express API. Deployed manually (no tracked CI pipeline for this — see Section 10).
Database	Supabase-managed PostgreSQL. The backend talks to it exclusively via the Supabase REST API using a service-role key (supabaseAdmin) — not a direct Postgres connection, and not Prisma (Prisma exists in the repo but is unused dead code).
Auth	Supabase Auth issues JWTs. The backend validates them per-request and cross-references a portal_users table for role/employee linkage.
File storage	Supabase Storage buckets (employee docs, client docs, invoices, timesheets, performance reviews, enrollment forms) — private by default, accessed via signed URLs.
Email	Azure Communication Services (ACS) — not Gmail/SMTP despite some stale comments elsewhere in the codebase; see Section 9.

3. Technology Stack

A consolidated reference of every technology in use, by layer. Deployment targets (Azure Static Web Apps, Azure Web App, GitHub Actions) are covered separately in Section 10 — this section is about what the application is built with, not where it runs.

3.1 Frontend

Technology	Role in the project
React 18	Core UI library — used for both the public site and the portal SPA
TypeScript 5	Static typing across the entire frontend
Vite 8 (Rolldown)	Dev server + production build tool; Rust-based Rollup replacement
React Router 6	Client-side routing for both apps, one shared BrowserRouter
TanStack Query 5	All server-state fetching/caching/mutations
Axios	HTTP client, wrapped by a custom apiClient with auth interceptors
Tailwind CSS 3	Utility-first styling for the portal (scoped under .portal-scope)
shadcn/ui + Radix UI primitives	~25 Radix packages (dialog, dropdown, tabs, tooltip, etc.) wrapped as shadcn/ui components in src/components/ui/
Zod + react-hook-form + @hookform/resolvers	Form schema validation and form state management
Recharts	All charts across Reports and analytics pages
Lucide React	Icon set used throughout the portal
TipTap	Rich text editor (email templates, announcements)
Sonner	Toast notification library
cmdk	Command palette component
Embla Carousel	Carousel/slider component in the portal UI kit

Technology	Role in the project
date-fns	Date formatting/manipulation utilities
Bootstrap + jQuery (legacy)	The public marketing site's original template stack, still active there only
Slick Carousel, Magnific Popup, animate.css	Public-site vendor plugins loaded via src/lib/vendors/

Frontend tooling / testing:

Technology	Role in the project
ESLint 9 + typescript-eslint	Linting
Vitest 4 + Testing Library	Unit/component tests
Playwright	End-to-end browser testing

3.2 Backend

Technology	Role in the project
Node.js + Express 4	HTTP server and routing
TypeScript 5	Static typing across the backend
Zod	Request validation schemas (backend/src/schemas/)
@supabase/supabase-js	Service-role Supabase client used for every database query
Helmet, CORS, express-rate-limit, compression	Security and traffic-shaping middleware
Morgan	HTTP request logging
Multer	Multipart file-upload handling
pdfkit	PDF generation (invoices, timesheets, reviews, enrollment forms)
docx + mammoth	Word document generation and parsing (monthly timesheet exports)
xlsx	Spreadsheet export support
sanitize-html	HTML sanitization for user-supplied rich text
node-cron	Scheduled jobs — invoice reminders, expiry sweeps (backend/src/jobs/scheduler.ts)
@azure/communication-email	Active email transport (Azure Communication Services)
dotenv	Local environment variable loading

Installed but unused at runtime — present in package.json but dead weight in the actual request path. Worth knowing before assuming either one is active:

Package	Why it's dead
Prisma (@prisma/client, prisma)	Installed and generated at build time, but zero runtime usage — every query goes through supabaseAdmin instead
nodemailer, resend	Installed from earlier mailer generations; the live mailer.ts uses Azure Communication Services exclusively

3.3 Database & Managed Platform Services

Service	Role in the project
PostgreSQL (via Supabase)	Primary relational database — 24+ tables, accessed exclusively through the Supabase REST API, never a direct Postgres connection
Supabase Auth	Issues and validates JWTs; auth.users is the identity root the portal_users table links into
Supabase Storage	Private file buckets for documents, invoices, timesheets, performance reviews, enrollment forms — always accessed via signed URLs

3.4 Version control & CI

Tool	Role in the project
Git / GitHub	Source control — single monorepo containing both frontend and backend
GitHub Actions	Automated frontend build + deploy on every push to main (no backend CI — see Section 10.2)

4. Public Marketing Site

The public site (`src/pages/`, `src/components/`) is a Bootstrap/jQuery-era HTML template converted into React components. It shares the same Vite build and React Router instance as the portal, but is architecturally and visually independent from it.

Route	File	Purpose
/	Index.tsx	Home page — hero slider, about, services, clients, stats, careers
/about	About.tsx	Company about page
/career-guidance	CareerGuidance.tsx	Career guidance content
/staffing-and-consulting	StaffingConsulting.tsx	Service detail page
/clients	Clients.tsx	Client showcase
/contact	Contact.tsx	Contact form
/technology	Technology.tsx	Technology/services page
/careers	Careers.tsx	Careers/job listings
*	NotFound.tsx	404 catch-all

Legacy jQuery integration

Vendor CSS (Bootstrap, Slick carousel, animate.css, icon fonts) lives in `src/styles/` and is imported in `src/main.tsx`, followed by `src/index.css` for custom overrides. jQuery plugins live in `src/lib/vendors/`. Because these are legacy non-module scripts, a custom Vite plugin — `injectjQueryGlobal()` in `vite.config.ts` — wraps each one to expose `window.$/window.jQuery` safely inside Vite's ES module context (proper UMD files like Bootstrap, Popper, Modernizr and Slick are skipped since they don't need it).

Icons use Font Awesome Free 5.x via CDN (only fas/far/fab — the fal set is Pro-only and unavailable). Layout uses Bootstrap grid utility classes (`d-none`, `d-xl-block`, `col-*`) — there is no Tailwind on the public site.

When overriding template styling, add new rules to `src/index.css` (loaded after `style.css`); if template specificity needs to be beaten directly, edit `src/styles/style.css` itself and add a matching mobile breakpoint rule.

5. Portal Application

The portal is a fully isolated React SPA mounted at `/portal/*`, lazy-loaded from the public site's entry point. It is wrapped in a `.portal-scope` class so its own Tailwind-based styling (`src/portal/portal.css`) never bleeds into the Bootstrap-based public site.

5.1 Page inventory

Roughly 60 distinct pages under `src/portal/pages/`:

Page	Purpose
Dashboard	Role router — renders the correct dashboard for the logged-in role
Employees / Employee Detail	Employee directory, filters, and full profile view
New Employee (wizard)	One component serves HR-create, HR-edit, self-onboarding, and self-edit (see 4.6)
People	Company-wide directory visible to all roles
HR drill-downs	Active / Onboarding / Inactive / Visa-Expiring / I-9-Status segmented employee views
Clients / Client Detail	Client account list and detail
Assignments / Assignment Detail	Employee-to-client staffing placements
Timesheets / Timesheet Detail	Weekly billing timesheets — submit/approve workflow
My Monthly Timesheet	Employee's own monthly attendance timesheet
Monthly Timesheets / Detail	HR/ops review of monthly attendance
Invoices / Invoice Detail / Invoice Editor	Client invoicing — list, detail, full create/edit editor
Record Payment	Full-page payment entry against an invoice
Public Invoice View	Token-gated, unauthenticated invoice viewing page
Invoice Analytics	Finance analytics on invoicing
Estimates	Client estimates/quotes
Reports	Central reports hub (Recharts-based, many sub-reports)
Documents	Document repository
Email Templates	Reusable templates for invoice-sending emails
Announcements	Company-wide announcement board
Expenses / Expense Analytics	Employee expense claims + finance analytics
Performance Reviews / Detail / My Reviews	HR-run appraisal cycles; employees view their own
Enrollment Forms / Detail	Benefits enrollment forms
Leave Requests	Leave/PTO application and approval workflow
Notifications	In-app notification center, click-to-navigate (see 4.7)
My Profile	Employee's own profile view

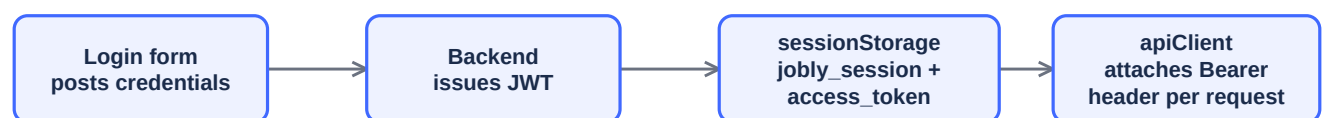
Page	Purpose
Admin Settings / System Settings	Admin-only configuration hubs
Audit Log	Admin-only audit trail viewer
Holidays Settings / Company Holidays	Holiday calendar configuration and viewing
Milestones / Headcount Report	HR milestone tracking and headcount analytics
Expiring Documents	HR view of documents nearing expiry
Workforce Availability / Client SLA	Operations availability planning and SLA tracking
Budgets / Cash Flow	Finance budgeting and cash-flow views
Tax Documents	Employee/finance tax document access
Assets	Company equipment inventory and assignment
Skills Registry	Employee skills catalog
Shift Schedule	Shift scheduling
Probation Tracker	New-hire probation period tracking
Capacity Report	Workforce capacity/utilization reporting
Onboarding Pending	"Awaiting HR review" screen after self-onboarding submission

5.2 Role-permission matrix

Enforced on the frontend by ProtectedRoute (allowedRoles) and mirrored server-side by requireRole(...) middleware on every route.

Feature area	Admin	HR	Ops	Finance	Employee
Dashboard / Announcements / People / Notifications / Timesheets	Y	Y	Y	Y	Y
Employees (list/detail)	Y	Y	Y		
Add / Edit Employee	Y	Y			
Clients / Invoices / Budgets / Cash Flow / Analytics	Y			Y	
Assignments	Y		Y		own
Attendance (self)	Y	Y			Y
Attendance Review	Y	Y	Y		
Leave Requests	Y	Y			Y
Expenses	Y	Y	Y	Y	Y
Performance Reviews (manage)	Y	Y			
My Reviews					Y
Enrollment Forms	Y	Y			Y
Documents	Y	Y			Y
Reports	Y	Y		Y	
My Profile					Y
Admin / System Settings / Audit Log	Y				
Milestones / Headcount / Skills / Probation	Y	Y			
Workforce / Shift Schedule / Capacity	Y	Y	Y		
Client SLA	Y		Y		
Tax Documents	Y			Y	Y
Estimates	Y			Y	
Assets	Y	Y	Y		

5.3 Authentication flow



Tokens are kept in `sessionStorage` (not `localStorage`), so a session clears when the tab closes. On a 401 response, the Axios response interceptor attempts one silent refresh via `POST /auth/refresh` before falling back to a forced logout/redirect. A 2-hour idle timer (mouse/keyboard/touch activity) also force-logs-out stale sessions. A 4-state onboarding gate (`in_progress` -> `pending_review` -> `changes_requested` -> `approved`) pins new hires to the onboarding wizard until HR approves them, enforced inside `ProtectedRoute`.

5.4 State management convention

All server state goes through TanStack Query, via a dedicated `QueryClient` (`src/portal/lib/queryClient.ts`). Every domain has a `useXxx` hook file in `src/portal/hooks/` (30 files) that defines a `mapXxx(row)` function translating the backend's `snake_case` row shape into the `camelCase` TypeScript interfaces in `src/portal/types/index.ts`. This is the single most important convention to know before touching any data field:

Rule: any new database column must be added in THREE places — the migration, the relevant `mapXxx()` function, and the TypeScript interface in `types/index.ts`. Missing any one of the three causes silent data loss on the frontend.

5.5 UI library & the custom Select component

The portal uses `shadcn/ui` (Radix primitives + Tailwind) throughout — 60+ reusable components in `src/components/ui/`. One notable exception: `src/components/ui/select.tsx` is a full in-house rewrite, no longer using `@radix-ui/react-select`. The original Radix component moved real DOM focus onto the open dropdown list, which — combined with its portaled, fixed-position rendering — caused the entire page to snap to the top on some browsers when reopening a dropdown with a pre-selected value. After exhausting configuration-level fixes, the team rewrote it using the ARIA "virtual focus" combobox pattern (tracking the highlighted option in React state via `aria-activedescendant` instead of moving real focus), which makes the bug structurally impossible rather than just patched. The public API is unchanged, so all ~40 existing call sites work without modification.

5.6 Onboarding wizard

`src/portal/pages/NewEmployee.tsx` is a single ~2,200-line component serving four distinct modes based on the current route, not four separate components:

Mode	Route	Behavior
Self-onboarding	<code>/portal/onboarding</code>	New hire completing their own profile; strict validation (LinkedIn, nationality, bank details, emergency-contact address all become required)
Self-edit	<code>/portal/profile (edit)</code>	Already-approved employee editing their own record
HR create	<code>/portal/employees/new</code>	HR/admin creating a new employee — only name + email required up front
HR edit	<code>/portal/employees/:id/edit</code>	HR/admin editing an existing employee's full record

Identity document uploads track an expiry date per document type. A validation rule (added after a real production bug) blocks submission whenever ANY document has an expiry date entered but no file actually

uploaded — not just the small set of hard-required document types — since recording an expiry for a document that was never provided is always a data-entry mistake.

5.7 Notification system

Notifications carry `entity_type/entity_id` plus a pre-computed `link` field. The link is computed and stored by the backend at creation time — not derived generically on the frontend — because the same entity type routes to a different page depending on the recipient's role (e.g. an "onboarding submitted" notification sends HR to the employee's detail page, but sends the employee themselves to their own profile). Clicking a notification (in the sidebar dropdown or the full Notifications page) marks it read and navigates to its link in one action.

6. Backend Architecture

The backend is a standalone Express 4 + TypeScript REST API (`backend/`), listening on port 3001, every route mounted under `/api/v1`. It is deployed independently of the frontend as a Docker container on Azure Web App.

6.1 Request pipeline



Every route follows this same chain: `authenticate -> requireRole(...roles) -> validateBody/validateQuery(schema) -> controller -> service -> supabaseAdmin`. Controllers stay thin (parse request, call service, shape the response); all business logic and every Supabase query lives in the service layer.

6.2 Domain / route map

~30 distinct resource domains, each with its own routes/controller/service files:

Domain	Base route(s)	Purpose
Auth	<code>/auth</code>	Login, logout, me, change-password, refresh, forgot-password
Employees	<code>/employees</code>	Employee CRUD, onboarding workflow, credentials, leave/termination, directory
Clients	<code>/clients</code>	Client company CRUD, billing contacts
Assignments	<code>/assignments</code>	Employee<->client project placements, bill/pay rates
Timesheets	<code>/timesheets</code>	Weekly billing timesheets + entries, approval state machine
Monthly Timesheets	<code>/monthly-timesheets</code>	Monthly attendance tracker, PDF/DOCX export
Leave Requests / Balance	<code>/leave-requests</code> , <code>/leave</code>	Leave applications + entitlement/balance tracking

Domain	Base route(s)	Purpose
Invoices	/invoices	Line items, discounts, payments, recurring templates, estimates, PDF/email
Invoice / Email Templates	/invoice-templates, /email-templates	PDF theme presets; reusable HTML email templates
Documents	/documents	Polymorphic file attachments via Supabase Storage
Enrollment Forms	/enrollment-forms	Benefits enrollment: HR creates, employee fills, PDF generated
Performance Reviews	/performance-reviews	Appraisal reports, 12-criteria rating grid, PDF + email
Notifications	/notifications	In-app notification fan-out, admin dedup view
Announcements	/announcements	Company-wide announcement board
Assets	/assets	Equipment inventory + assignment
Expenses	/expenses	Expense report submission/approval
Budgets	/budgets	Department budgets vs. actuals
Shifts	/shifts	Shift scheduling per employee/date
Skills	/skills	Employee skills directory
Holidays	/holidays	Company holiday calendar
Onboarding / Offboarding Checklist	/onboarding-checklist, /offboarding-checklist	Configurable task templates + per-employee instances
Tax Documents	/tax-documents	W-2/1099-style document tracking
Activity Logs	/activity-logs	Audit trail viewer
Analytics / Reports	/analytics, /reports	Dashboards and exports across employees, timesheets, invoices
Nav Badges	/nav-badges	Sidebar unread/pending counters
Settings / System / Admin	/portal-settings, /system, /admin	System config, ops endpoints, password resets, user management
Public	/public	Unauthenticated endpoints — contact form, public invoice view

6.3 Auth & RBAC internals

Identity chain: `auth.users` (Supabase-managed) <-> `portal_users` (role, `employee_id`) <-> `employees` (full HR record). A portal user's role isn't cached in the JWT — the `authenticate` middleware re-fetches the `portal_users` row on every request, so role/permission changes take effect immediately, not on next login. `requireRole(...roles)` is called roughly 140 times across the route files — the authoritative permission check lives server-side even though the frontend also gates navigation for UX purposes.

6.4 Validation pattern

26 Zod schema files in `backend/src/schemas/`, one per resource, each exporting typed input schemas (e.g. `CreateEmployeeInput`). Routes wrap the schema with `validateBody(schema)` / `validateQuery(schema)` (`backend/src/middleware/validate.ts`) — a failed parse raises a 400 `ValidationError` with a readable field-by-field message, handled centrally in `errorHandler.ts`.

7. Database Schema

The database is Supabase-managed PostgreSQL. The backend never uses a direct Postgres connection or Prisma at runtime (both exist in the repo but are unused/dead code) — every query goes through the Supabase REST API via a service-role client (`supabaseAdmin`, `backend/src/config/supabase.ts`). Row Level Security is enabled on tracked tables as a defense-in-depth backstop (a blanket service-role-only policy) — the real authorization boundary is the application code (`requireRole` + service-layer filtering), not the database.

7.1 Tables defined in tracked migrations

27 numbered, additive-only SQL files in `backend/supabase/migrations/` (001 -> 027):

Table(s)	Purpose
<code>portal_users</code>	Login identity — 1:1 with <code>auth.users</code> , carries role + <code>employee_id</code>
<code>employees</code>	Core HR record — personal, address, employment, immigration, pay, onboarding state
<code>clients</code>	Client company + billing contact info
<code>assignments</code>	Employee<->client project placement, rates, status
<code>timesheets</code> / <code>timesheet_entries</code>	Weekly billing timesheet header + per-day hours
<code>invoices</code> / <code>invoice_line_items</code> / <code>invoice_timesheets</code>	Client invoices, line items, and the timesheet<->invoice junction
<code>documents</code>	Polymorphic file metadata pointing at Supabase Storage
<code>notifications</code>	Per-recipient in-app notifications (<code>entity_type/entity_id/link</code>)
<code>activity_logs</code>	Audit trail — actor, action, entity, metadata
<code>monthly_timesheets</code>	Independent monthly attendance timesheet (JSONB day entries)
<code>leave_requests</code>	Prospective leave applications + approval workflow

Table(s)	Purpose
leave_types / employee_leave_entitlements	Configurable leave categories + per-employee grants
products / payments / recurring_invoice_templates	Invoice catalog items, manual payments, recurring-invoice config
email_templates / invoice_templates	Reusable HTML email templates; PDF theme presets
announcements	Company announcement board
performance_reviews	Appraisal reports (12-criteria JSONB rating grid)
enrollment_forms	New-hire benefits enrollment form (JSONB form_data)

7.2 Important gap — tables that exist live but aren't tracked

The following tables are actively queried by the backend service layer but have NO corresponding CREATE TABLE statement anywhere in backend/supabase/migrations/: assets, expense_reports, department_budgets, company_holidays, tax_documents, offboarding_templates / offboarding_tasks, onboarding_checklist_templates / employee_onboarding_tasks, employee_skills, shifts, system_settings. They were evidently created directly against the Supabase project (dashboard or SQL editor) outside the versioned migration history. Anyone extending these features should verify the live column list directly against the Supabase project rather than trusting the migrations folder alone — it is not a complete source of truth for the current schema.

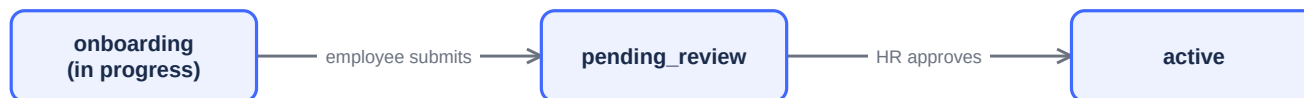
7.3 Key relationships

employees.reporting_manager_id is a self-referencing FK for the management chain. assignments links employees to clients. timesheets are unique per (employee, assignment, week) and link to invoice_line_items via the invoice_timesheets junction table when billed. documents uses a polymorphic entity_type/entity_id pointer (employee | client | invoice) rather than a hard foreign key. notifications.user_id points at portal_users, with its own polymorphic entity_type/entity_id plus the pre-computed link field described in Section 5.7.

8. Core Business Workflows

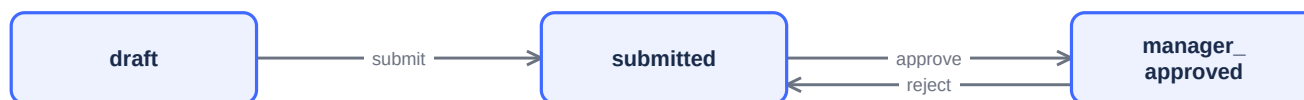
8.1 Onboarding state machine

employees.status is one of onboarding / active / inactive. "Pending review" and "changes requested" are NOT separate enum values — they're derived from whether onboarding_completed_at and onboarding_change_request_message are set. A shared checklist function (backend/src/lib/onboarding.ts, computeOnboarding()) is the single source of truth for completion %, used both to gate the employee's own submit action and HR's approval action.



HR can instead "request changes," which clears `onboarding_completed_at` and routes the employee back to the wizard with a message — a documented resubmit loop rather than a hard rejection.

8.2 Timesheet state machine



Only these transitions are valid — enforced in code, not just at the database level. Submitting a zero-hour week requires a `leave_reason`; hours can't be logged on a day the employee already has an approved leave request (checked at both submit and manager-approve time, since leave can be approved after a timesheet is already submitted). Admin is exempt from these business gates.

8.3 Notification fan-out & dedup

`createNotification()` inserts one row per recipient — a "notify all HR" event creates N rows, one per HR user, each carrying the same title/message/entity/link. For the admin's notification view specifically, these fan-out copies are collapsed back into one logical event (matched by title+message+entity), tracked via a separate `admin_read` column so an admin's own read state never collides with the actual recipient's read flag.

9. Integrations

9.1 Email — Azure Communication Services

`backend/src/lib/mailer.ts` sends all transactional email exclusively through Azure Communication Services (@azure/communication-email), configured via `AZURE_COMM_CONNECTION_STRING` and `ACS_SENDER_ADDRESS`. A `mailerConfigured` flag guards every send call so a missing config never crashes a request — it just skips the email and logs it. Every email uses a shared branded HTML shell with the Jobly logo.

Note: some older comments elsewhere in the codebase (including the root `CLAUDE.md`) still describe the mailer as `nodemailer` + Gmail SMTP or Brevo SMTP — those describe earlier generations of the mailer that were since replaced. The `nodemailer/resend` packages remain installed but are unused. Always verify against `mailer.ts`'s actual imports, not surrounding comments.

9.2 PDF / DOCX generation

backend/src/lib/pdfGenerator.ts (pdfkit for PDFs, the docx package for Word output) has six generator entry points: invoices, monthly timesheets (PDF and DOCX), yearly timesheets, performance reviews, and enrollment forms. The pattern is consistent everywhere it's used: generate a Buffer in memory -> upload it to a Supabase Storage bucket -> create a 7-day signed URL -> persist that URL on the record's pdf_url column.

9.3 Supabase Auth & Storage

Login uses supabaseAnon.auth.signInWithPassword(); everything else server-side goes through supabaseAdmin with a pre-seeded service-role Authorization header (a documented workaround for a supabase-js client bug that could otherwise silently drop it). Storage buckets are private by default and always accessed through signed URLs, never public links, except one inferred public bucket for employee profile photos.

10. Deployment & Infrastructure

10.1 Frontend — automatic

Every push to the main branch triggers a GitHub Actions workflow (.github/workflows/azure-static-web-apps-*.yaml) that builds the Vite app (output_location: dist) and deploys it to Azure Static Web Apps via the official Azure/static-web-apps-deploy action. No manual step required.

```
git push origin main
# -> GitHub Actions builds + deploys automatically
gh run list --limit 1          # check status
gh run watch <run-id> --exit-status  # wait for completion
```

10.2 Backend — manual runbook

There is no tracked CI workflow for the backend in this repository. Deployment to Azure Web App is a manual process — documented here as the exact working sequence.

```
# 1. Build and push the Docker image (capture the sha256 digest printed)
docker buildx build --no-cache --platform linux/amd64 \
  -t prashanth1710/jobly-backend:latest --push backend/

# 2. Point the Web App at the exact new digest
az webapp config set -n <app-name> -g <resource-group> \
  --linux-fx-version "DOCKER|prashanth1710/jobly-backend@sha256:<DIGEST>"

# 3. Restart and wait for /health
az webapp restart -n <app-name> -g <resource-group>
curl -sf https://<app-host>/health # expect {"status":"ok", ...}
```

Important: the container is digest-pinned, not tag-pinned — pushing a new :latest tag alone does nothing; the linux-fx-version must be re-pointed to the new digest explicitly. Always keep the previous known-good digest on hand before deploying, and health-poll after every restart — a build that exits 0 can still produce a crash-looping container.

10.3 Environment configuration

Every variable actually read by the running code, grouped by whether it's required, actively used, or present but dead. This is the list to work from when standing up a new environment — not backend/.env.example, which is out of date (see the callout below).

Frontend — set at build time, baked into the bundle

Variable	Purpose
VITE_API_URL	The backend API base URL. Read once at build time (import.meta.env), not at runtime — changing it requires a rebuild + redeploy, not just an env change on a running server.

Gotcha: the production GitHub Actions workflow never sets VITE_API_URL — there's no build-time env step for it. When unset, src/portal/lib/apiClient.ts falls back to a backend URL that's hardcoded directly in that source file. In practice this means the deployed frontend's backend URL is currently pinned in source code, not configured via environment. If the backend's hostname ever changes, update the fallback in apiClient.ts (or add a real VITE_API_URL build step to the workflow) and redeploy the frontend — an env var change alone won't reach it.

Backend — required (the process exits at boot if these are missing)

Variable	Purpose
SUPABASE_URL	The Supabase project's API URL
SUPABASE_SERVICE_ROLE_KEY	Service-role key — bypasses RLS, used for every database query
SUPABASE_ANON_KEY	Anon key — used only for the login call (supabaseAnon.auth.signInWithPassword)
PORT	Defaults to 3001 if unset
NODE_ENV	development / production / test — defaults to development

Backend — actively used, not boot-enforced

Variable	Purpose
FRONTEND_URL	Used for the CORS allowlist and for building deep-links in outgoing emails. Defaults to http://localhost:8080 if unset — must be set to the real frontend URL in production or CORS will reject it
AZURE_COMM_CONNECTION_STRING	The active email transport (Azure Communication Services). Without it, mailerConfigured is false and every email is silently skipped (logged, not thrown)
ACS_SENDER_ADDRESS	The verified ACS sender address emails are sent from
ENABLE_SCHEDULER	Must be exactly "true" to enable the daily cron job (invoice reminders, expiry sweeps). Off by default — safe to leave unset on any non-primary environment to avoid duplicate scheduled sends
CONTACT_TO	Recipient for the public contact form. Defaults to info@joblysolutions.com if unset

Backend — present but not part of the live code path

Variable(s)	Why it's inert
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS / MAIL_FROM	Validated as optional by the Zod env schema, but mailer.ts never reads them — leftover from an earlier SMTP-based mailer generation
GMAIL_USER / GMAIL_APP_PASSWORD	Same — leftover from an even earlier Gmail-based mailer generation
HR_FALLBACK_EMAIL	Validated as optional but not read by any current code path
DATABASE_URL / DIRECT_URL	Only consulted by the unused Prisma client (src/lib/prisma.ts) and the npx prisma CLI — the running app never opens a direct Postgres connection

backend/.env.example itself is stale: it documents only PORT, NODE_ENV, the three SUPABASE_* variables, DATABASE_URL/DIRECT_URL, FRONTEND_URL, and GMAIL_USER/GMAIL_APP_PASSWORD — it doesn't mention AZURE_COMM_CONNECTION_STRING, ACS_SENDER_ADDRESS, ENABLE_SCHEDULER, or CONTACT_TO at all, and still frames Gmail as the email option. Use the tables above, not that file, when setting up a new environment — this is the same class of "comments lag behind the real code" gap already flagged for the mailer in Section 9.1.

What actually changes when standing up a new environment (e.g. staging, or moving to a new Azure subscription): a new Supabase project means new SUPABASE_URL/SUPABASE_SERVICE_ROLE_KEY/SUPABASE_ANON_KEY on the backend (and re-running every migration against it); a new backend hostname means updating FRONTEND_URL on the backend AND the hardcoded fallback in apiClient.ts on the frontend; a new frontend hostname means updating the backend's FRONTEND_URL for CORS to keep working; a new email sender means new AZURE_COMM_CONNECTION_STRING/ACS_SENDER_ADDRESS.

10.4 Database migrations

The Supabase project's direct database host is IPv6-only, which blocks local psql access from most Windows setups — migrations are applied through the Supabase MCP tooling's apply_migration action (or the Supabase dashboard SQL editor) rather than a local CLI. New migration files are still added to backend/supabase/migrations/ for history, numbered sequentially, additive-only (ADD COLUMN IF NOT EXISTS style) — no down-migrations exist in this project.

11. How to Add a New Feature

This is the concrete, repeatable pattern this codebase already uses for every existing feature. Following it keeps a new feature consistent with everything else in the system.

- Database** — add a new migration file in backend/supabase/migrations/ (numbered sequentially, ADD COLUMN IF NOT EXISTS / CREATE TABLE IF NOT EXISTS style). Apply it via the Supabase tooling, not a local psql connection.
- Validation** — add or extend a Zod schema in backend/src/schemas/ for the new input shape.
- Service** — add the Supabase query logic in the relevant backend/src/services/*.service.ts file (or a new one for a new domain), using supabaseAdmin.

- 4 **Controller + route** — add a thin controller function and wire it into the router with the correct middleware chain: `authenticate -> requireRole(...roles) -> validateBody(schema) -> controller`.
- 5 **Frontend hook** — add/extend a `useXxx` hook in `src/portal/hooks/`, including a `mapXxx()` function that translates the backend's `snake_case` response into `camelCase`.
- 6 **Types** — update the corresponding TypeScript interface in `src/portal/types/index.ts` to match exactly what `mapXxx()` now returns.
- 7 **UI** — build the page/component using the existing `shadcn/ui` component set in `src/components/ui/` and the shared portal components in `src/portal/components/shared/` wherever a suitable one already exists.
- 8 **Routing** — register the new page in `src/portal/PortalApp.tsx` inside a `ProtectedRoute` with the correct `allowedRoles` for who should see it.
- 9 **Navigation** — add a sidebar entry in `src/portal/components/layout/PortalSidebar.tsx` if the feature needs its own nav item, scoped to the same roles as the route.
- 10 **Notifications (if relevant)** — if the feature should notify anyone, call `createNotification(userId, title, message, type, entityType, entityId, link)` — compute the link server-side, accounting for the recipient's role if the same entity type routes differently per role (see Section 5.7).
- 11 **Verify end-to-end** — `npx tsc --noEmit`, `npm run build (frontend)`; backend `npx tsc --noEmit`; a local or staging smoke test of the new flow before deploying.

Appendix A — Key File Reference Index

Quick lookup for "where do I start" when making a common kind of change:

If you need to...	Start here
Add/change a portal page	<code>src/portal/pages/</code> , register route in <code>src/portal/PortalApp.tsx</code>
Add/change a backend endpoint	<code>backend/src/routes/</code> , <code>backend/src/controllers/</code> , <code>backend/src/services/</code>
Change validation rules	<code>backend/src/schemas/*.schema.ts</code>
Change a DB table/column	<code>backend/supabase/migrations/</code> + the matching <code>mapXxx()</code> in <code>src/portal/hooks/</code>
Change role permissions	<code>requireRole(...)</code> in backend routes; <code>allowedRoles</code> in <code>PortalApp.tsx</code> / <code>PortalSidebar.tsx</code>
Change email content	<code>backend/src/lib/mailer.ts</code>
Change PDF layout (invoices etc.)	<code>backend/src/lib/pdfGenerator.ts</code>
Change notification behavior	<code>backend/src/services/notifications.service.ts</code> , <code>src/portal/hooks/useNotifications.ts</code>
Change auth/session behavior	<code>backend/src/middleware/auth.ts</code> , <code>src/portal/context/AuthContext.tsx</code> , <code>src/portal/lib/apiClient.ts</code>
Change shared UI components	<code>src/components/ui/</code> (<code>shadcn/ui</code>), <code>src/portal/components/shared/</code>
Change public site content	<code>src/pages/</code> , <code>src/components/</code> (non-ui)

Appendix B — Glossary

Term	Meaning
displayId vs UUID	Every entity has both a UUID primary key (used for API calls/foreign keys) and a human-readable displayId like EMP-0001, INV-2026-0001 (used only in the UI).
Onboarding states	in_progress -> pending_review -> (changes_requested <-> pending_review) -> approved. Derived from employees columns, not a dedicated enum.
Timesheet states	draft -> submitted -> manager_approved, with rejection paths back to submitted.
RBAC	Role-Based Access Control — the 5 roles (admin, hr, operations, finance, employee) that gate both frontend routes and backend endpoints.
mapXxx() / useXxx()	The frontend convention: every hook maps the backend's snake_case response into a camelCase TypeScript object.
entity_type / entity_id	A polymorphic reference pattern used by documents and notifications to point at any kind of record (employee, client, invoice, etc.) without a rigid foreign key.
supabaseAdmin	The service-role Supabase client used for every backend database query — bypasses Row Level Security by design; all authorization happens in application code instead.